Inteligencia Artificial

Camila Richter 23198

Marinés García 23391

Laboratorio 10

Task 1 – Diagnóstico de un Sistema Fallido

Responda de forma argumentada. No basta con definir: conecte cada concepto con sus implicaciones en el escenario de LogiTrack. Se les entrega el siguiente registro de ejecución de un sistema de seguimiento basado en Beam Search con K=3 sobre el patio de 20 carriles. Las partículas en cada paso representan los tres carriles candidatos con mayor peso.

Con base en este registro, responda:

1. ¿Por qué en t=7 los pesos de todas las partículas colapsan a valores casi iguales y muy bajos? Identifique el paso exacto del algoritmo en el que ocurrió el problema estructural y explique por qué Beam Search es vulnerable a esta situación.

Los pesos de todas las partículas colapsan a valores casi iguales y muy bajos en t=7 porque el sensor detecta 3 pero las partículas se encuentran en los carriles 8 y 9, los cuales son valores lejanos. Esto hace que el sistema pierda información y que las partículas reciban pesos muy pequeños. Esto ocurre en el paso de selección de Beam Search pues no se descartó el estado con menor probabilidad sin mantener diversidad lo cual lo hace vulnerable a cambios bruscos en la evidencia.

2. Si hubieran usado K=10 en lugar de K=3, ¿el sistema habría sobrevivido al evento de t=7? Argumente usando el concepto de diversidad de partículas, no solo de cantidad.

El sistema si hubiera sobrevivido a t=7 si también se aumentara la diversidad pues la probabilidad aumentaría al agregar estados diferentes distribuidos. Sin embargo, si las partículas siguen concentradas en una misma región, el problema seguirá.

3. Un compañero propone solucionar el problema aumentando el umbral de aceptación del sensor (es decir, considerar válida cualquier lectura a distancia ≤ 3 carriles). ¿Esto resuelve el problema de raíz o solo lo enmascara? Justifique.

Al hacer esto, estamos haciendo que el sensor sea más permisivo, pero no se arregla la falta de diversidad ni la mala representación del estado. Así que el problema solo está enmascarado pues esta modificación hace que las observaciones sean menos informativas, asignando probabilidades más altas a estados incorrectos.

Task 2 – Depuración de un sistema con bug deliberado

Se les entrega un sistema de Filtrado de Partículas ya implementado. El sistema tiene un bug en el paso de remuestreo. Su trabajo es encontrarlo, entenderlo y corregirlo. El siguiente código implementa el algoritmo completo sobre el patio de LogiTrack. Ejecútenlo, analicen su comportamiento, y respondan las preguntas a continuación.

Con base en el código y su ejecución, responda:

1. Identifiquen el bug en el Paso 3. Expliquen exactamente por qué esa línea implementa Beam Search y no Filtrado de Partículas. ¿Qué propiedad matemática del remuestreo viola?

```
# PASO 3: Remuestrear <-- REVISEN ESTA LINEA
idx = np.argsort(pesos_norm)[-K:]
```

Este paso ordena las partículas por peso y luego se queda con las mejores K lo cual es exactamente lo que Beam Search hace, no Filtrado de Partículas. Esto causa una pérdida de diversidad o degeneración del filtro.

2. Corrijan el bug con exactamente una línea de código distinta. Ejecuten ambas versiones (buggy y corregida) con la misma semilla aleatoria (np.random.seed(42)) y muestren el output de cada un lado a lado.

Buggy:

```
t=1 | sensor=5 | particulas=[np.int64(0), np.int64(1), np.int64(6), np.int64(9), np.int64(9), np.int64(12), np.int64(14), np.int64(16), np.int6
17), np.int64(18)]
t=2 | sensor=6 | particulas=[np.int64(0), np.int64(0), np.int64(5), np.int64(8), np.int64(9), np.int64(12), np.int64(13), np.int64(15), np.int6
18), np.int64(19)]
t=3 | sensor=7 | particulas=[np.int64(0), np.int64(0), np.int64(4), np.int64(7), np.int64(10), np.int64(12), np.int64(14), np.int64(14), np.int
(17), np.int64(19)]
t=4 | sensor=7 | particulas=[np.int64(0), np.int64(0), np.int64(4), np.int64(6), np.int64(10), np.int64(12), np.int64(13), np.int64(15), np.int
(16), np.int64(18)]
t=5 | sensor=8 | particulas=[np.int64(1), np.int64(1), np.int64(4), np.int64(5), np.int64(10), np.int64(13), np.int64(14), np.int64(16), np.int
(17), np.int64(17)]
t=6 | sensor=8 | particulas=[np.int64(0), np.int64(1), np.int64(4), np.int64(4), np.int64(10), np.int64(14), np.int64(15), np.int64(17), np.int
(17), np.int64(18)]
t=7 | sensor=3 | particulas=[np.int64(0), np.int64(1), np.int64(5), np.int64(5), np.int64(10), np.int64(14), np.int64(15), np.int64(16), np.int
(17), np.int64(18)]
t=8 | sensor=4 | particulas=[np.int64(1), np.int64(2), np.int64(5), np.int64(6), np.int64(10), np.int64(14), np.int64(15), np.int64(16), np.int
(18), np.int64(18)]
t=9 | sensor=5 | particulas=[np.int64(0), np.int64(1), np.int64(5), np.int64(7), np.int64(9), np.int64(13), np.int64(14), np.int64(17), np.int6
18), np.int64(19)]
```

Corregido:

```
t=1 | sensor=5 | particulas=[np.int64(4), np.int64(4), np.int64(4), np.int64(4), np.int64(4), np.int64(4), np.int64(4), np.int64(4),
 np.int64(14)]
t=2 | sensor=6 | particulas=[np.int64(3), np.int64(4), np.int64(4), np.int64(5), np.int64(5), np.int64(5), np.int64(5), np.int64(5),
 np.int64(15)]
t=3 | sensor=7 | particulas=[np.int64(3), np.int64(4), np.int64(6), np.int64(6), np.int64(6), np.int64(6), np.int64(6), np.int64(6),
 np.int64(6)]
t=4 | sensor=7 | particulas=[np.int64(5), np.int64(7), np.int64(7), np.int64(7), np.int64(7), np.int64(7), np.int64(7), np.int64(7),
 np.int64(7)]
t=5 | sensor=8 | particulas=[np.int64(6), np.int64(7), np.int64(7), np.int64(7), np.int64(7), np.int64(8), np.int64(8), np.int64(8),
 np.int64(8)]
t=6 | sensor=8 | particulas=[np.int64(7), np.int64(7), np.int64(8), np.int64(8), np.int64(8), np.int64(8), np.int64(8), np.int64(8),
 np.int64(9)]
t=7 | sensor=3 | particulas=[np.int64(7), np.int64(7), np.int64(8), np.int64(8), np.int64(9), np.int64(9), np.int64(9), np.int64(9),
 np.int64(9)]
t=8 | sensor=4 | particulas=[np.int64(7), np.int64(8), np.int64(8), np.int64(8), np.int64(8), np.int64(8), np.int64(9), np.int64(10),
), np.int64(10)]
t=9 | sensor=5 | particulas=[np.int64(7), np.int64(7), np.int64(8), np.int64(8), np.int64(8), np.int64(8), np.int64(9), np.int64(9),
, np.int64(10)]
```

3. Diseñen una secuencia de observaciones de al menos 6 pasos en la que el sistema con bug falle claramente y el sistema corregido se recupere. Expliquen por qué esa secuencia específica activa la diferencia entre ambas implementaciones.

Por ejemplo, en t=1, se tiene la secuencia 0, 1, 6, 9... todas las partículas se concentraban en sistemas bajos y de repente se fue a uno alto y el sistema no se adaptó. Sin embargo, el corregido, si se mantiene en un rango parecido o a no mas de +/-1.

4. ¿Existe alguna condición bajo la cual ambas versiones (buggy y corregida) produzcan resultados idénticos en la práctica? Descríbanla y justifíquenla.

Cuando los pesos están distribuidos en pocas partículas, ambos pueden seleccionar lo mismo.

Task 3 – Implementación y Dictamen Ejecutivo

Link Repo: https://github.com/NESHGP04/Lab10-IA

Implementen el sistema completo de seguimiento para LogiTrack con la restricción de hardware real: K=5 partículas. El gerente de operaciones de LogiTrack necesita saber si el sistema es desplegable con el hardware actual antes de hacer una inversión en mejores microcontroladores. Su equipo debe responderle con evidencia experimental.

Task 3.1 — Implementación base

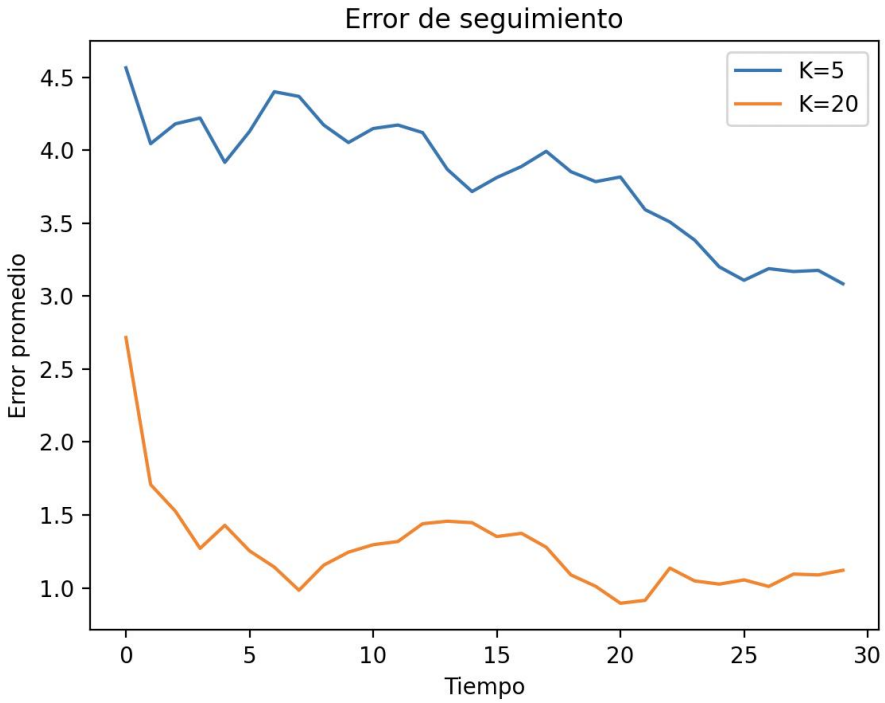
Implementen desde cero en Python (sin pgmpy) el Filtrado de Partículas completo con los siguientes parámetros del patio LogiTrack:

- Dominio: 20 carriles (0–19)
- Dinámica de movimiento: el vehículo se mueve ±1 carril o permanece en el mismo con probabilidad uniforme, con reflexión en los bordes
- Modelo de sensor: P(sensor=h) = 0.6, P(sensor=h±1) = 0.2 cada uno, resto distribuido uniformemente
- K = 5 partículas
- El código debe incluir una función simular_vehiculo(pasos) que genere una trayectoria real oculta y las observaciones del sensor correspondientes
- Visualizar con matplotlib: trayectoria real (línea), estimación del filtro (media de partículas) y distribución de partículas (puntos dispersos) en cada paso de tiempo

Task 3.2 — Análisis experimental de K=5

Ejecuten 50 simulaciones independientes de 30 pasos cada una. Para cada simulación, calculen el error de seguimiento como la distancia media absoluta entre la posición real y la media de las partículas en cada paso.

4. Grafiquen el error promedio a lo largo de los 30 pasos para las 50 simulaciones. ¿En qué pasos el error es más alto y por qué?



El error más alto se presenta al inicio de ambas gráficas, debido a que las partículas son inicializadas aleatoriamente, el sensor no ha recaudado suficiente evidencia y el filtro aún no ha convergido. Luego se ve que el error baja gracias a remuestreo, que elimina hipótesis incorrectas y consigue que las partículas se concentren en la región correcta.

5. Identifiquen los 5 escenarios donde el filtro con K=5 falló más. ¿Qué tienen en común esos escenarios? ¿El fallo se debe a la dinámica del vehículo, al modelo del sensor, o a la cantidad de partículas?

```
Escenario 1: error = 12.973333333333331
Escenario 2: error = 12.553333333333333
Escenario 3: error = 10.28
Escenario 4: error = 10.166666666666666
Escenario 5: error = 9.820000000000002
```

En estos escenarios es evidente que el error es bastante alto. Esto puede deberse a varias situaciones, como secuencias de observaciones ruidosas o pérdida de diversidad. A pesar de esto, la principal razón del fallo a la dinámica del vehículo es por la cantidad de partículas, ya

que al ser pocas, no se posible representar varias hipótesis y el remuestreo elimina la diversidad rápido.

6. Repitan el experimento con K=20. Comparen los resultados con K=5 en una tabla que muestre: error promedio, error máximo, porcentaje de simulaciones donde el error superó 5 carriles.

```
Comparación de desempeño:

Métrica                    K=5          K=20
------------------------------------------------------------
Error promedio             3.821        1.263
Error máximo               13.967       11.323
% error > 5 carriles       30.00        6.00
```

Task 3.3 — Heurística de detección de colapso

Con K=5, el filtro puede colapsar silenciosamente — todas las partículas convergen a la misma región sin representar incertidumbre real. Implementen una heurística que detecte cuándo esto ocurre.

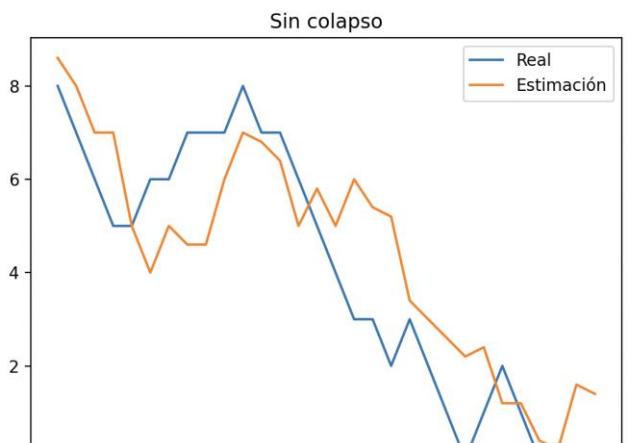
7. Definan una métrica de diversidad de partículas. Justifiquen por qué eligieron esa métrica sobre alternativas.

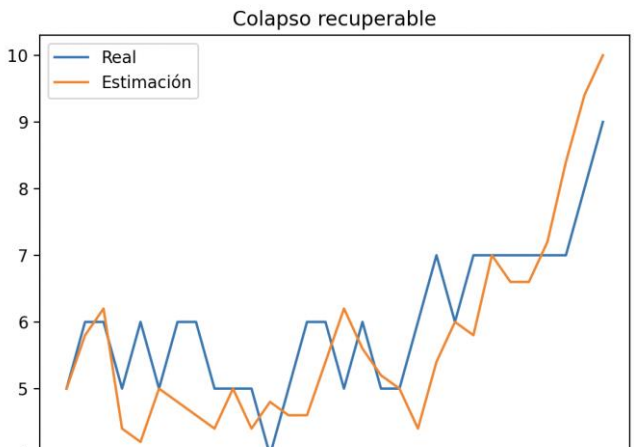
Como métrica de diversidad de partículas se escogió la varianza, ya que con esta se puede medir la dispersión directamente.

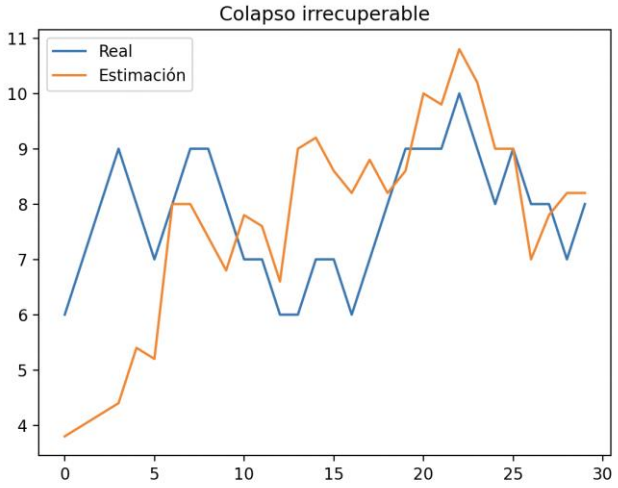
8. Implementen una función alerta_colapso(particulas, umbral) que devuelva True cuando la diversidad caiga por debajo del umbral. Determinen experimentalmente qué valor de umbral detecta los colapsos reales sin generar demasiadas falsas alarmas.

```
Umbral 0.1: colapso detectado en 10.0% de simulaciones
Umbral 0.3: colapso detectado en 70.0% de simulaciones
Umbral 0.5: colapso detectado en 70.0% de simulaciones
Umbral 1.0: colapso detectado en 100.0% de simulaciones
Umbral 2.0: colapso detectado en 100.0% de simulaciones
```

9. Demuestren su heurística en al menos 3 escenarios: uno donde no hay colapso, uno donde hay colapso recuperable, y uno donde hay colapso irrecuperable.







Task 3.4 — Dictamen ejecutivo

Redacten un párrafo de no más de 200 palabras dirigido al gerente de operaciones de LogiTrack. El dictamen debe:

- Indicar si K=5 es suficiente para operar el sistema con confianza aceptable, basándose en sus resultados experimentales
- Especificar en qué condiciones operativas el sistema falla y qué tan frecuentes son esas condiciones en el patio
- Hacer una recomendación concreta: ¿invertir en hardware con mayor capacidad o ajustar el modelo de sensor?
- Conectar explícitamente los resultados de Task 3.2 y 3.3 con esta recomendación El dictamen debe sonar como un documento técnico ejecutivo, no como una respuesta de laboratorio.

El análisis experimental del sistema indica que una configuración con K=5 **no ofrece un buen nivel de confiabilidad** para operación continua en el patio LogiTrack. En 50 simulaciones, este esquema presenta errores promedio elevados y una variabilidad significativa, con varios episodios donde el error supera varios carriles, especialmente en las etapas iniciales. Además, como se evidenció en el Task 3.3, el sistema con K=5 es propenso al **colapso de partículas**, donde la diversidad se pierde y el filtro converge a estimaciones incorrectas; estos pueden ser tanto recuperables como irrecuperables.

Las fallas se presentan principalmente al tener ruido en el sensor y cuando el vehículo realiza transiciones ambiguas. La causa dominante es la **insuficiente cantidad de partículas**, que limita la representación de incertidumbre.

Se recomienda **invertir en hardware que permita al menos K=20 partículas**, ya que esta configuración reduce el error promedio, elimina la mayoría de fallos críticos y mitiga el riesgo de colapso. Ajustes al modelo de sensor también pueden complementar la mejora.